

Design and Analysis of Algorithms

Lecture 7: Greedy Algorithms





Table of Contents

- Optimization Problems
- Optimization Algorithms
- One of the optimization Algorithms: Greedy Algorithms



Optimization Problems

- **Optimization Problems** are the **problems themselves** - the mathematical formulations of real-world scenarios where we want to find the "best" solution according to some criteria.
- **Structure of an Optimization Problem:**
 - ✓ **Decision Variables** (what we can control)
 - ✓ **Objective Function** (what we want to maximize/minimize)
 - ✓ **Constraints** (limitations and requirements)



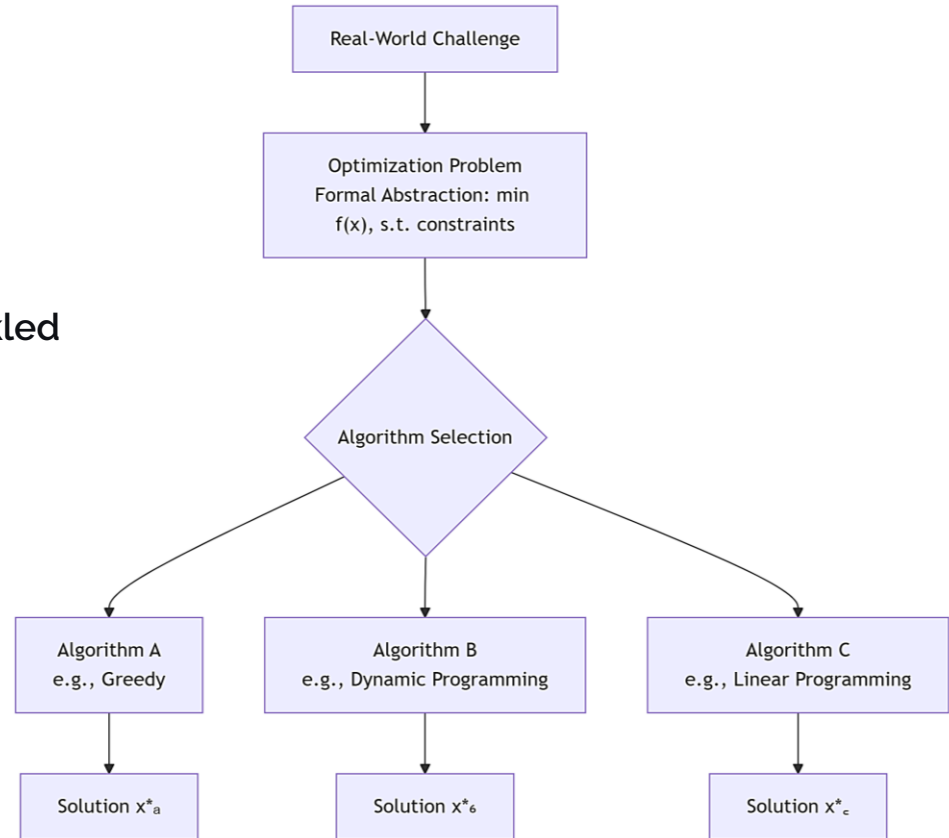
Optimization Algorithms

- An Optimization Algorithm is a systematic, step-by-step computational procedure designed to solve an optimization problem.
- Key Properties of an Optimization Algorithm:
 - ✓ **Correctness:** Does it produce a feasible solution? Does it find the true optimum?
 - ✓ **Convergence:** Will it eventually find a solution given infinite time?
 - ✓ **Efficiency:** How do time and memory requirements scale with problem size (Computational Complexity)?
 - ✓ **Termination:** How does it know when to stop?



Optimization Algorithms

A single optimization problem can often be tackled by multiple, distinct optimization algorithms.



Greedy Algorithms

- A greedy algorithm is a simple, intuitive algorithmic paradigm that builds up a solution piece by piece. At each step, it makes the choice that seems most optimal at that very moment. **In other words, it makes a locally optimal choice in the hope that this choice will lead to a globally optimal solution.**
- The key principle is: "Take the best you can get right now, without regard for future consequences."



General Structure of a Greedy Algorithm

- **Most greedy algorithms follow a similar pattern:**
 - ✓ **Candidate Set:** Start with a set of all possible choices (e.g., all edges in a graph, all coins for change).
 - ✓ **Selection Function:** Choose the "best" candidate from the set according to a greedy criterion (e.g., the smallest weight, the highest value-to-weight ratio).
 - ✓ **Feasibility Check:** Check if the chosen candidate, when added to the solution, is feasible (e.g., doesn't violate a constraint).
 - ✓ **Solution Check:** Determine if the current set of chosen candidates constitutes a complete solution. If yes, stop. If not, continue.



Some Applications of Greedy Algorithms

- Scheduling Problem
- Fractional Knapsack Problem
- Job Sequencing
- Minimum Spanning Tree



The University Scheduling Department needs to efficiently allocate its limited classroom resources among various academic classes. Each class has a fixed start time and end time, and classes requiring the same classroom cannot overlap in time.

Given a list of class intervals with their start and end times, determine the maximum number of non-overlapping classes that can be scheduled in a single classroom, or equivalently, find the minimum number of classrooms required to accommodate all classes.



The Naïve Algorithm:
Brute Force

Greedy algorithms



The classroom scheduling problem using Brute Force algorithm

- **Classroom Scheduling: "Which Classes to Choose?"**

Decision: Include or exclude each class from the schedule.

Order doesn't matter - we only care about the **set** of classes.

Binary choice per class: Take it or leave it.

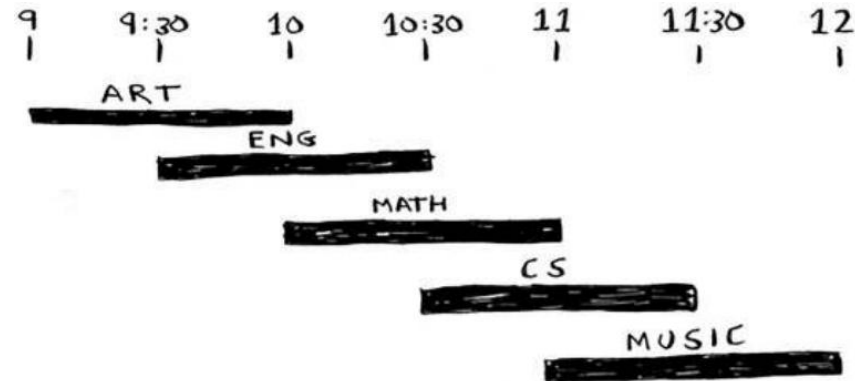
- Brute force complexity: $O(2^n)$

```
FUNCTION simple_brute_force(classes):  
    best = [] // Will store our best schedule  
  
    // Try including/excluding each class in all possible ways  
    FOR include_A in [True, False]:  
        FOR include_B in [True, False]:  
            FOR include_C in [True, False]:  
                // ... repeat for all classes  
  
                current = []  
                IF include_A: ADD class_A to current  
                IF include_B: ADD class_B to current  
                IF include_C: ADD class_C to current  
                // ... for all classes  
  
                IF no_overlaps(current):  
                    IF length(current) > length(best):  
                        best = current  
  
    RETURN best
```

The classroom scheduling problem using a greedy algorithm

- Suppose you have a classroom and want to hold as many classes here as possible. You get a list of classes as shown in the first top figure.
- You can't hold all of these classes in there, because some of them overlap, as shown in the down figure.

CLASS	START	END
ART	9 AM	10 AM
ENG	9:30 AM	10:30 AM
MATH	10 AM	11 AM
CS	10:30 AM	11:30 AM
MUSIC	11 AM	12 PM



The classroom scheduling problem using a greedy algorithm

- You want to hold as many classes as possible in this classroom. How do you pick what set of classes to hold, so that you get the biggest set of classes possible?
- Sounds like a hard problem, right? Actually, the algorithm is so easy, it might surprise you. Here's how it works:
 - ✓ **Pick the class that ends the soonest. This is the first class you'll hold in this classroom.**
 - ✓ **Now, you have to pick a class that starts after the first class. Again, pick the class that ends the soonest. This is the second class you'll hold.**
 - ✓ **Keep doing this, and you'll end up with the answer!**



The classroom scheduling problem using a greedy algorithm

- Let's try it out. Art ends the soonest, at 10:00 a.m., so that's one of the classes you pick.
- Now you need the next class that starts after 10:00 a.m. and ends the soonest.
- English is out because it conflicts with Art, but Math works. Finally, CS conflicts with Math, but Music works.

ART	9 AM	10 AM	✓
ENG	9:30 AM	10:30 AM	
MATH	10 AM	11 AM	
CS	10:30 AM	11:30 AM	
MUSIC	11 AM	12 PM	

(1)

ART	9 AM	10 AM	✓
ENG	9:30 AM	10:30 AM	X
MATH	10 AM	11 AM	✓
CS	10:30 AM	11:30 AM	
MUSIC	11 AM	12 PM	

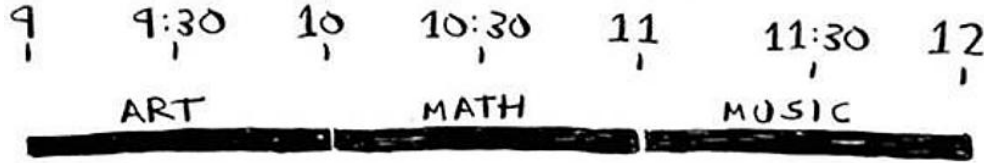
(2)

ART	9 AM	10 AM	✓
ENG	9:30 AM	10:30 AM	X
MATH	10 AM	11 AM	✓
CS	10:30 AM	11:30 AM	X
MUSIC	11 AM	12 PM	✓

(3)

The classroom scheduling problem using a greedy algorithm

- So, these are the three classes you'll hold in this classroom.



- A greedy algorithm is simple: at each step, pick the optimal move. In this case, each time you pick a class, you pick the class that ends the soonest.
- In technical terms: at each step you pick the locally optimal solution, and in the end, you're left with the globally optimal solution.
- Believe it or not, this simple algorithm finds the optimal solution to this scheduling problem!



The classroom scheduling problem using a greedy algorithm

- The greedy algorithm for classroom scheduling has a time complexity of $O(n \log n)$ because of its two main steps. First, it must sort all the classes by their end time. Sorting n items using an efficient algorithm like Merge Sort or Quick Sort always takes $O(n \log n)$ time; this is a well-known computational cost for organizing any list of items through comparisons. Second, the algorithm makes a single pass through this now-sorted list to pick non-overlapping classes, which only takes $O(n)$ time.
- When we combine these steps, the total time is **$O(n \log n)$** + **$O(n)$** . For a large number of classes, the **$O(n \log n)$** sorting step dominates and determines the overall complexity, making it much more efficient than a brute-force approach.

```
FUNCTION schedule_classes(classes):  
    IF classes is empty:  
        RETURN empty list  
  
    // Step 1: Sort classes by ending time  
    SORT classes BY end_time ASCENDING  
  
    // Step 2: Initialize with first class  
    selected_classes = [first_class]  
    last_end_time = first_class.end_time  
  
    // Step 3: Greedily select non-overlapping classes  
    FOR each class in remaining_classes:  
        IF class.start_time >= last_end_time:  
            ADD class to selected_classes  
            last_end_time = class.end_time  
  
    RETURN selected_classes
```

A thief breaks into a store with a knapsack that can carry a maximum weight capacity W . The store contains n items, each with a specific weight w_i and value v_i . The thief wants to maximize the total value of stolen items without exceeding the knapsack's weight capacity.

The Naïve Algorithm:
Brute Force

Greedy algorithms



The knapsack problem

- Given a set of items, each with a weight and a value, determine a subset of items to include in a collection so that the total weight is less than or equal to a given limit and the total value is as large as possible.
- The knapsack problem is in combinatorial optimization problem. It appears as a subproblem in many, more complex mathematical models of real-world problems.
- **Applications:**
 - ✓ In many cases of resource allocation along with some constraint, the problem can be derived in a similar way of Knapsack problem. Following is a set of example.
 - ✓ Finding the least wasteful way to cut raw materials
 - ✓ Cutting stock problems



The knapsack problem

- Suppose there is a thief is robbing a store and can carry a maximal weight of W into his knapsack.
- There are n items available in the store and weight of i item is w_i and its profit is p_i . What items should the thief take?
- In this context, the items should be selected in such a way that the thief will carry those items for which he will gain maximum profit.
- Hence, the objective of the thief is to maximize the profit.
- Based on the nature of the items, Knapsack problems are categorized as: **Fractional Knapsack** and **0/1 Knapsack**.



Fractional knapsack problem using brute force algorithm

- **Brute Force:** Guaranteed optimal but exponentially slow
- **Time Complexity:** $O(2^n)$

```
FUNCTION brute_force_knapsack(weights, values, capacity):  
    n = length(weights)  
    max_value = 0  
  
    # Generate all possible subsets (2^n possibilities)  
    FOR bitmask FROM 0 TO (2^n - 1):  
        total_weight = 0  
        total_value = 0  
  
        # Check which items are included in this subset  
        FOR i FROM 0 TO n-1:  
            IF bitmask has i-th bit set:  
                total_weight += weights[i]  
                total_value += values[i]  
  
        # If valid and better, update best solution  
        IF total_weight <= capacity AND total_value > max_value:  
            max_value = total_value  
  
    RETURN max_value
```

Fractional knapsack problem using a greedy algorithm- Example (1)

- You have a bag that can hold **maximum 50kg**. You want to fill it with items to get the **maximum total value**. You can take fractions of items. The available items shows in the following table.

Item	Value	Weight
A	60	10kg
B	100	20kg
C	120	30kg



Fractional knapsack problem using a greedy algorithm- Example (1)

- **Step 1: Calculate Value per Kg**

✓ For each item, find: **Value $\div$ Weight**

Item	Value	Weight	Value/Weight
A	60	10kg	6.0
B	100	20kg	5.0
C	120	30kg	4.0



Fractional knapsack problem using a greedy algorithm- Example (1)

- **Step 2: Sort by Value/Weight (Highest First)**

Item	Value/Weight	Value	Weight
A	6.0	60	10kg
B	5.0	100	20kg
C	4.0	120	30kg



Fractional knapsack problem using a greedy algorithm- Example (1)

- **Step 3: Fill the Bag in Order**

✓ Bag capacity = 50kg

Step	Item	Decision	Weight	Value Gained	Remaining Capacity
1	A	Take all	10kg	60	40kg
2	B	Take all	20kg	100	20kg
3	C	Take $20/30 = 2/3$	30kg	80	0kg



Fractional knapsack problem using a greedy algorithm- Example (1)

- **Step 4: Final Result**

Total Weight	Total Value
$10 + 20 + 20 = \mathbf{50kg}$	$\mathbf{240} = 80 + 100 + 60$



Fractional knapsack problem using a greedy algorithm- Example (1)

- **Time Complexity**

Step	Operation	Time Complexity	Why
1	Calculate ratios	$O(n)$	One operation per item
2	Sort items	$O(n \log n)$	Comparison-based sorting
3	Greedy selection	$O(n)$	Single pass-through items
4	Final calculation	$O(1)$	Simple arithmetic

- **Total time: $O(n \log n)$ - dominated by sorting**



Fractional knapsack problem using a greedy algorithm- Example (2)

- A cargo airline needs to optimize the loading of its aircraft to maximize revenue per flight. The aircraft has a **maximum payload capacity of 50 metric tons**. The airline must select the optimal combination of available cargo shipments to maximize total revenue while respecting weight constraints. Available Cargo Shipments is listed in the table below.

Cargo Type	Revenue	Weight
High-Value Electronics	60,000\$	10 tons
Pharmaceuticals	100,000\$	20 tons
Industrial Machinery	120,000\$	30 tons



Fractional knapsack problem using a greedy algorithm- Example (2)

- **Step 1: Calculate Revenue per weight**

Cargo Type	Revenue per weight
High-Value Electronics	$60,000/10 = 6,000$
Pharmaceuticals	$100,000/20 = 5,000$
Industrial Machinery	$120,000/30 = 4,000$



Fractional knapsack problem using a greedy algorithm- Example (2)

- **Step 2: Sort by Revenue per weight**

Cargo Type	Revenue per weight
High-Value Electronics	6,000
Pharmaceuticals	5,000
Industrial Machinery	4,000



Fractional knapsack problem using a greedy algorithm- Example (2)

- Step 3: Strategic Aircraft Loading**

Loading Sequence	Cargo Type	Loading Decision	Weight Loaded	Revenue Generated	Remaining Capacity
1	High-Value Electronics	Take all	10 tons	60,000\$	40 tons
2	Pharmaceuticals	Take all	20 tons	100,000\$	20 tons
3	Industrial Machinery	Take 20/30	30 tons	80,000\$	0 tons



Fractional knapsack problem using a greedy algorithm- Example (2)

- **Step 4: Final Result**

Total Weight	Total Revenue
$10 + 20 + 20 = \mathbf{50kg}$	$80 + 100 + 60 = 240,000$

- **Total time complexity: $O(n \log n)$ - dominated by sorting**



A company has n jobs to process, each with:

A deadline d_i (by which it must be completed)

A profit p_i (earned if completed by deadline)

Only one job can be processed at a time, and each job takes exactly 1 unit of time. The goal is to maximize total profit by selecting a sequence of jobs that meet their deadlines.



The Naïve Algorithm:
Brute Force

Greedy algorithms



Job Sequencing with Deadline using brute force algorithm

- The brute force approach tries every possible order of jobs.
- For each ordering, it checks if jobs can be completed before their deadlines when processed in that sequence. If valid, it calculates the total profit. After checking all possible sequences, it returns the maximum profit found.
- With n jobs, there are $n!$ (n factorial) possible orders. For just 10 jobs, that's 3,628,800 sequences to check!



Job Sequencing with Deadline using greedy algorithm- Example (1)

- Let us consider a set of jobs as shown in the following table. We need to find a sequence of jobs that can be completed within their deadlines and yields maximum profit. Each job is associated with a deadline and profit.

Job	Deadline	Profit
J ₁	2	60
J ₂	1	100
J ₃	3	20
J ₄	2	40
J ₅	1	20



Job Sequencing with Deadline using greedy algorithm- Example (1)

- **Step 1: Sort Jobs by Profit (Descending Order)**

Job	Profit	Deadline
J ₂	100	1
J ₁	60	2
J ₄	40	2
J ₃	20	3
J ₅	20	1



Job Sequencing with Deadline using greedy algorithm- Example (1)

- **Step 2: Initialize Time Slots**

Maximum deadline = 3, so we have 3 time slots: [1, 2, 3]

- **Step 3: Job Assignment Process**

Job J_2 (Profit: 100, Deadline: 1)

- Assign to latest available slot ≤ 1 : **Slot 1**
- Timeline: [J_2 , _ , _]

Job J_1 (Profit: 60, Deadline: 2)

- Assign to latest available slot ≤ 2 : **Slot 2**
- Timeline: [J_2 , J_1 , _]



Job Sequencing with Deadline using greedy algorithm- Example (1)

- **Step 3: Job Assignment Process**

Job J₄ (Profit: 40, Deadline: 2)

- Check slots ≤ 2 : Slot 2 occupied, Slot 1 occupied
- **Cannot assign** - skip J₄

Job J₃ (Profit: 20, Deadline: 3)

- Assign to latest available slot ≤ 3 : **Slot 3**
- Timeline: [J₂, J₁, J₃]

Job J₅ (Profit: 20, Deadline: 1)

- Check slots ≤ 1 : Slot 1 occupied
- **Cannot assign** - skip J₅



Job Sequencing with Deadline using greedy algorithm- Example (1)

- **Step 4: Final Solution**

Scheduled Jobs: J_2, J_1, J_3

Total Profit: $100 + 60 + 20 = 180$

Final Timeline:

Time Slot	1	2	3
Job	J_2	J_1	J_3



Job Sequencing with Deadline using greedy algorithm- Example (1)

- The job sequencing algorithm uses **nested loops**:
- **Outer Loop:**
 - ✓ Iterates through all n jobs (after sorting)
 - ✓ Processes each job one by one in profit-descending order
- **Inner Loop:**
 - ✓ For each job, finds the latest available time slot before its deadline
 - ✓ May need to check up to d time slots (where d is the maximum deadline)
- **Time Complexity: $O(n^2)$**



Job Sequencing with Deadline using greedy algorithm- Example (2)

- A single machine needs to schedule 6 jobs. Each job has a processing time of 1 unit, a deadline, and a profit. The goal is to select jobs to maximize total profit while meeting all deadlines.

Job	Deadline	Profit
J ₁	3	50
J ₂	1	30
J ₃	4	70
J ₄	2	40
J ₅	5	90
J ₆	2	60



Job Sequencing with Deadline using greedy algorithm- Example (2)

- **Step 1: Sort Jobs by Profit (Descending Order)**

Job	Profit	Deadline
J ₅	90	5
J ₃	70	4
J ₆	60	2
J ₁	50	3
J ₄	40	2
J ₂	30	1



Job Sequencing with Deadline using greedy algorithm- Example (2)

- **Step 2: Initialize Time Slots**

Maximum deadline = 5, so we have 5 time slots: [1, 2, 3, 4, 5]

Initial Timeline: [_, _, _, _, _]



Job Sequencing with Deadline using greedy algorithm- Example (2)

• Step 3: Job Assignment Process

Job J₅ (Profit: 90, Deadline: 5)

- Assign to latest available slot ≤ 5 : **Slot 5**
- Timeline: [_, _, _, _, J₅]
- Profit: 90

Job J₃ (Profit: 70, Deadline: 4)

- Assign to latest available slot ≤ 4 : **Slot 4**
- Timeline: [_, _, _, J₃, J₅]
- Profit: 90 + 70 = 160

Job J₆ (Profit: 60, Deadline: 2)

- Assign to latest available slot ≤ 2 : **Slot 2**
- Timeline: [_, J₆, _, J₃, J₅]
- Profit: 160 + 60 = 220



Job Sequencing with Deadline using greedy algorithm- Example (2)

• Step 3: Job Assignment Process

Job J₁ (Profit: 50, Deadline: 3)

- Assign to latest available slot ≤ 3 : Slot 3
- Timeline: `[_, J6, J1, J3, J5]`
- Profit: $220 + 50 = 270$

Job J₄ (Profit: 40, Deadline: 2)

- Check slots ≤ 2 : Slot 2 occupied, Slot 1 available
- Assign to Slot 1
- Timeline: `[J4, J6, J1, J3, J5]`
- Profit: $270 + 40 = 310$

Job J₂ (Profit: 30, Deadline: 1)

- Check slots ≤ 1 : Slot 1 occupied
- Cannot assign - skip J₂



Job Sequencing with Deadline using greedy algorithm- Example (2)

- **Step 4: Final Solution**

Scheduled Jobs: J_4, J_6, J_1, J_3, J_5

Jobs Not Scheduled: J_2

Total Profit: $40 + 60 + 50 + 70 + 90 = 310$

- **Time Complexity: $O(n^2)$**

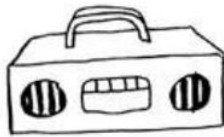


While greedy algorithms successfully solve problems such as job sequencing, classroom scheduling, and fractional knapsack optimally, they surprisingly fail for other optimization problems such as the 0/1 knapsack and finding the largest sum problems. The key difference lies in whether immediate optimal choices guarantee a global optimum or actually prevent it.



0/1 knapsack problem

- Suppose you're a greedy thief. You're in a store with a knapsack, and there are all these items you can steal. But you can only take what you can fit in your knapsack. The knapsack can hold 35 pounds, as shown in the above figure.
- Suppose there are three items you can steal, as following.



STEREO
\$3000
30lbs



LAPTOP
\$2000
20lbs

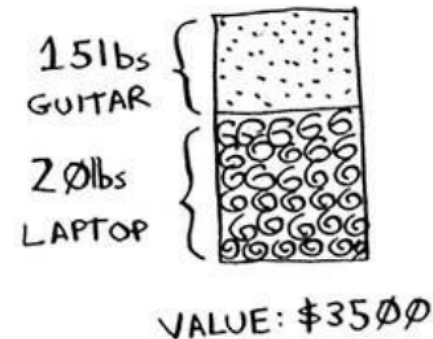
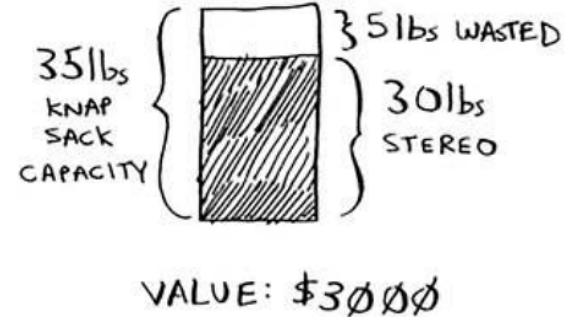


GUITAR
\$1500
15lbs



0/1 knapsack problem

- Your knapsack can hold 35 pounds of items. The stereo system is the most expensive, so you steal that. Now you don't have space for anything else, as shown in the top figure.
- You got \$3,000 worth of goods. But wait! If you'd picked the laptop and the guitar instead, you could have had \$3,500 worth of loot, as shown in the second down figure!



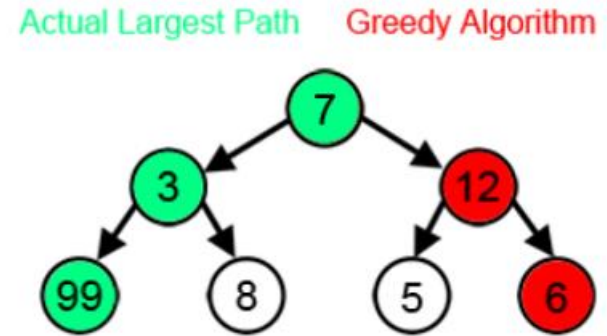
0/1 knapsack problem

- Clearly, the greedy strategy doesn't give you the optimal solution here. But it gets you pretty close.
- Here's the takeaway from this second example: sometimes, perfect is the enemy of good.
- Sometimes all you need is an algorithm that solves the problem pretty well. And that's where greedy algorithms shine, because they're simple to write and usually get pretty close.
- Greedy algorithms are a good choice because not only are they simple to come up with, but that simplicity means they usually run fast, too.



Finding the largest sum

- Here, the goal is to reach the largest sum. At every stage, the greedy algorithm will select the largest optimal solution.
- Beginning from 7 as the initial point, in the first step, the algorithm will select 12 instead of 3. This will take the algorithm far from the correct solution since it will not reach 99.
- That indicates that the greedy algorithm may not necessarily reach the global optimal solution by selecting the immediate local optimal solutions.



Any Question?

